A DEVELOPMENT JOURNAL

The Painted *Ascent*

Prologue



GIOVANNI MANTEGNA · Unreal Engine 5.8 · 2026

OVERVIEW

A small angel in a *dark cave*

The Painted Ascent started from my own drawings of a little stone angel, half statue and half child. Over three months I turned those drawings into a playable prologue, on my own, learning most of it along the way.



The first room: a light beam, a puzzle of crystals, and a door that will not open yet.

The prologue takes place in a cave of glowing crystals. Step by step the angel learns something new: to interact with the world, to jump twice, to dash through the air, to punch, to be launched into the air, to strike with all its strength. The hints stay short, because each obstacle is built to have a single, obvious answer.

At the end of the cave, in the dark, a brush and a palette are waiting. The angel takes them, the walls turn purple, and the first real fight begins.



The arena: the Colossus and its Casters, waiting for the angel to make the first move.

Why I made it

I come from engineering and artificial intelligence, and I want to work in games. This project was my way to find out whether I could take an idea all the way from a sketch to something people can actually play: design the level, build the characters, write every system in Blueprints, and make it all feel like one small world.

3 months JUNE – SEPTEMBER 2026	**Solo** DESIGN, ART, CODE, UI	**Unreal Engine 5.8** BUILT IN BLUEPRINTS
3 rooms 8 SECTIONS	**2 enemies** COLOSSUS AND CASTER	**1 cinematic** MADE IN SEQUENCER

Inside this journal

1 Game and level design

2 Art pipeline

3 Animation

4 Gameplay programming

5 Effects, light and interface

6 AI in my workflow

7 Looking back

The prologue is complete and playable from start to finish. Sound and music are still to come.
Watch the gameplay: **youtube.com/watch?v=6CvP8dkcaxc**

One step *at a time*

Three rooms, split into eight short sections. Each section introduces one ability, and the obstacle in front of you has a single, obvious answer.

One answer

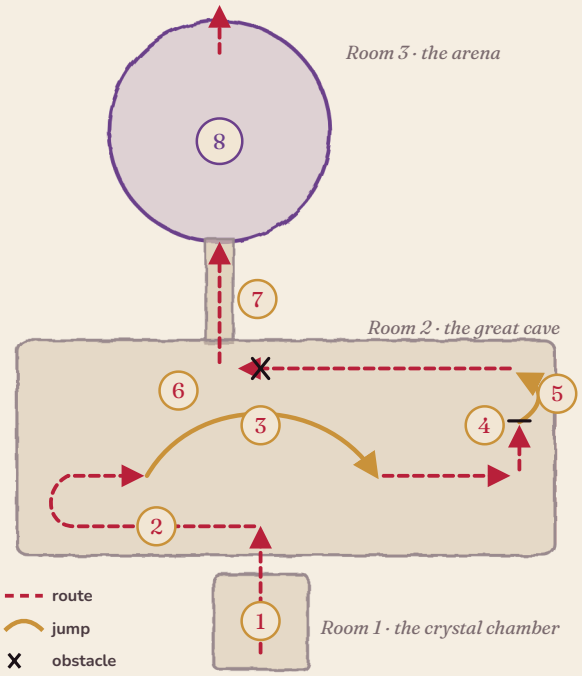
Each obstacle is solved in one clear way.

One thing at a time

An ability is learned on its own before it is combined with others.

Shape, not signs

The geometry forces the mechanic instead of simply allowing it.



1. **Crystal puzzle.** Rotate the crystals to link them with beams of light: when the chain is complete, the door opens.
2. **Rolling rocks.** Burning rocks roll towards you: jump over them.
3. **The gap.** A jump boost and the dash carry you across.
4. **Red stone.** One punch breaks it.
5. **Launch.** Wait where the stone was: the floor throws you up to the ledge.
6. **Charged punch.** Collect ink, fill the seal, break the last rock.
7. **Corridor.** Nothing to learn, on purpose: a breath before the fight.
8. **Arena.** Everything together.



Section 2: burning rocks on the path.



Section 4: the red stone.



Glowing drops along a narrow ledge.

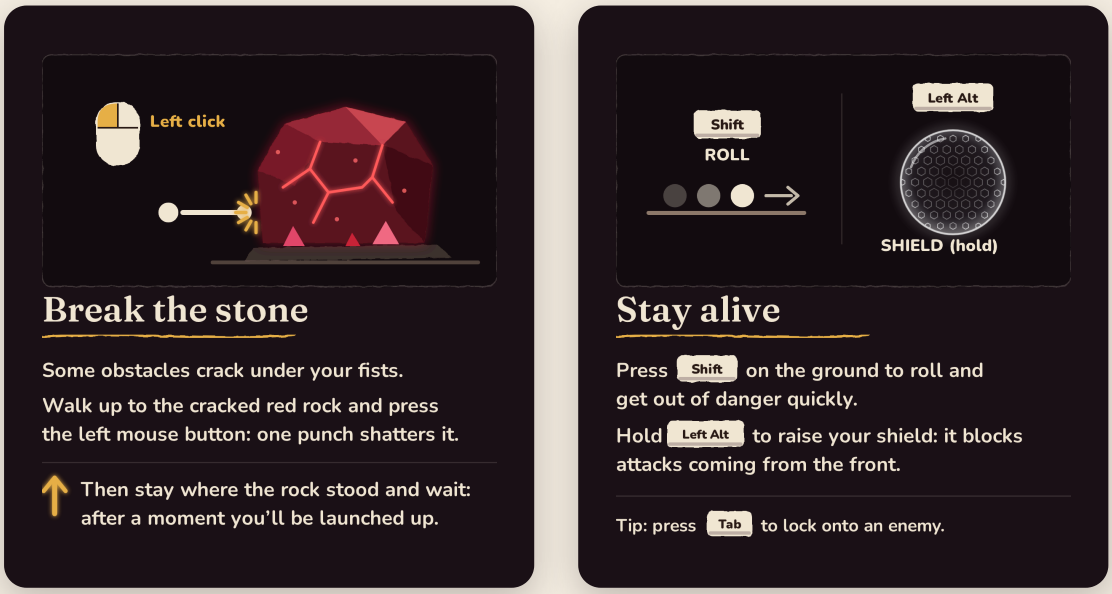


Section 7: the corridor, weapons waiting at the end.

Teaching without interrupting

Reaching certain spots unlocks a short illustrated card. You can reopen all the cards you have seen at any time with T. In the arena the enemies wait until you close the combat cards: the fight starts when you are ready.

Dying sends you back to the last section reached, with everything you solved still solved. The cinematic can be skipped, and after a death it is not shown again.

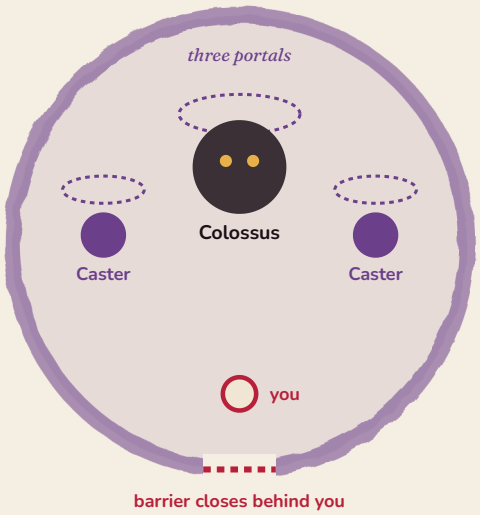


Two of the hint cards, as they appear in the game.

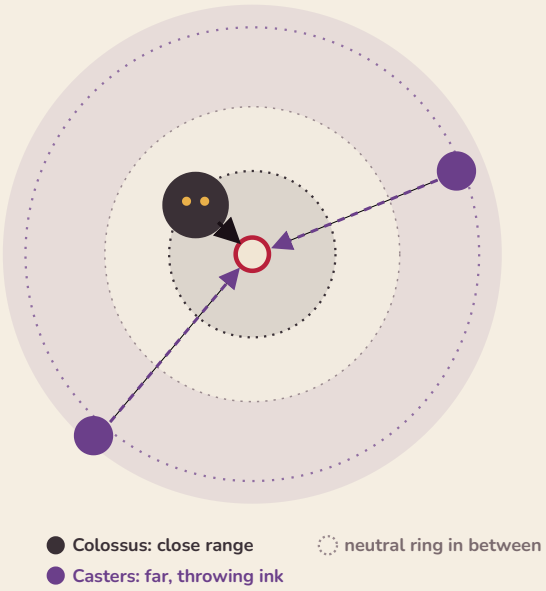
The arena

My reference was the combat of *Kingdom Hearts 2*: at every moment you should understand what each enemy is doing, and there should be no dead time where nothing readable happens. The fight is built on two enemies with opposite roles.

At the start



During the fight



The Colossus

Slow, heavy and impossible to interrupt. You cannot trade blows with it: you have to move around it and punish it after its attacks.

The Casters

Fragile, and they keep their distance while throwing ink. Each one you defeat leaves a drop of ink behind, and a new one comes out of a portal.

Phase 1

Colossus 100% → 50%

Casters keep coming back, ink can be refilled.

Shield

Colossus protected

Only a charged strike with the brush breaks it. It costs three drops: a full seal means you are ready.

Phase 2

Colossus 50% → 0%

No more Casters. Ink runs out.

Where it stands

I did not fully reach the goal. The fight works and can be finished, but some moments still leave an enemy's intention unclear, and at times the rhythm drops. It is the first thing I would go back to (chapter 7).

From pencil to *paint*

Every model in the game starts as one of my drawings. AI tools turn the drawing into a 3D base; I choose the input, compare the tools, and finish the result by hand.

1	2	3	4	5	6
Sketch	**FLUX.2 [klein]**	**Tripo AI**	**Hunyuan3D**	**PixelPasto 2.0**	**GIMP**
pencil or digital	ComfyUI, local	3D model	base texture	Blender, painted look	colour tuning

The angel



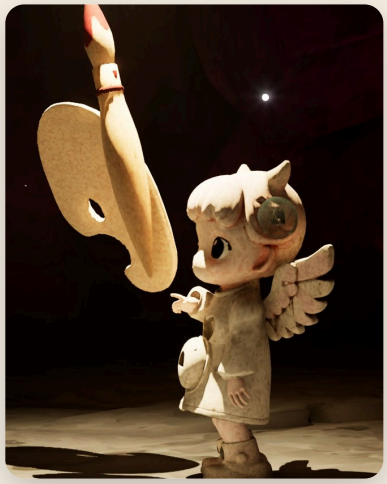
The pencil sketch.

→



FLUX.2 [klein]: a clean image, ready for 3D.

→



In the game.

One piece at a time

To get clean models out of Tripo AI, I split the character into parts (head, body, dress, wings, hands, shoes) and generated each one separately, then put them back together in Blender.

I also tried **TRELLIS 2** locally on my RTX 4070, but the results were not as good as **Tripo AI**, the paid service I kept.



The head, generated on its own.

Props, the same way

The props follow the same road: a quick digital sketch, a clean image with FLUX.2 [klein], then Tripo AI.



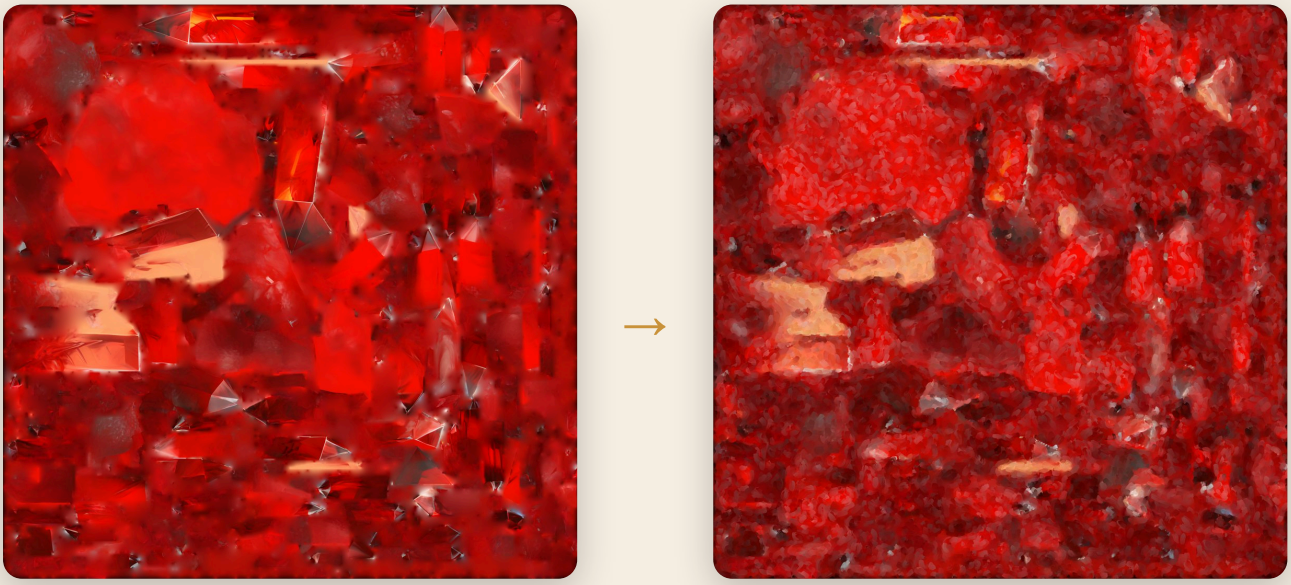
Two props: the sketch, then the image sent to Tripo AI.

Clean topology, recently possible

Until recently, AI-generated meshes were dense and messy, hard to rig and animate. That changed only in the last months: first Hunyuan3D added automatic retopology, then Tripo AI released **Smart Mesh**, which generates models with clean topology. Generating the character in parts and joining them in Blender, every piece comes out ready to use.

The painted look

The textures are what make the game look painted, and they come in two steps. **Hunyuan3D** (free, by Tencent) creates a base texture from the image and the 3D model. **PixelPasto 2.0**, a paid Blender setup (a .blend file), then repaints it with visible brush strokes. When the colours drifted, I corrected hue and temperature in GIMP.



Base texture from Hunyuan3D.

After PixelPasto 2.0.

Every asset in the game went through this treatment. That shared brush stroke is what holds the world together.

Bringing them to *life*

Most of the angel's animations come from free packs on Fab and from Unreal's library. My work was making them fit a body they were never made for.

Rig and skin

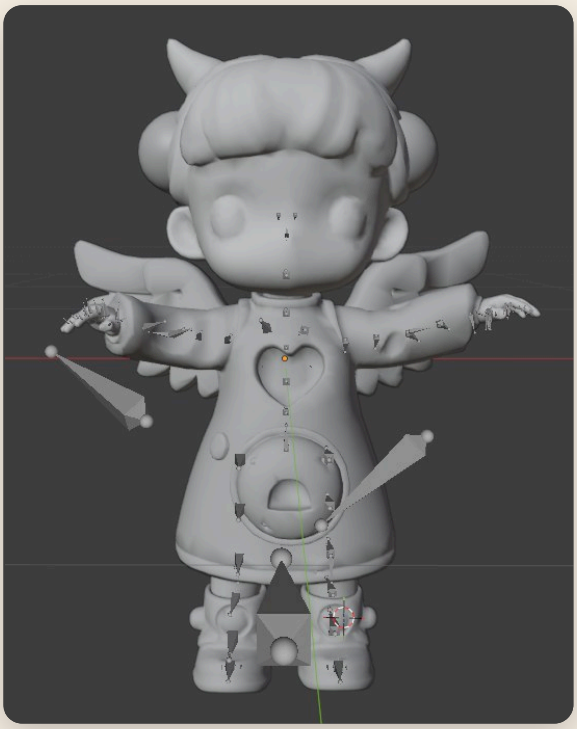
Rigged with **AccuRig**, weights painted by hand. The dress stays rigid, like carved stone; clipping was fixed on the model itself.

Retargeting

The animations were made for Unreal's mannequin. I set up the retargeting to the angel's chibi proportions, fixing the mapping that made it float and teleport.

Animation logic

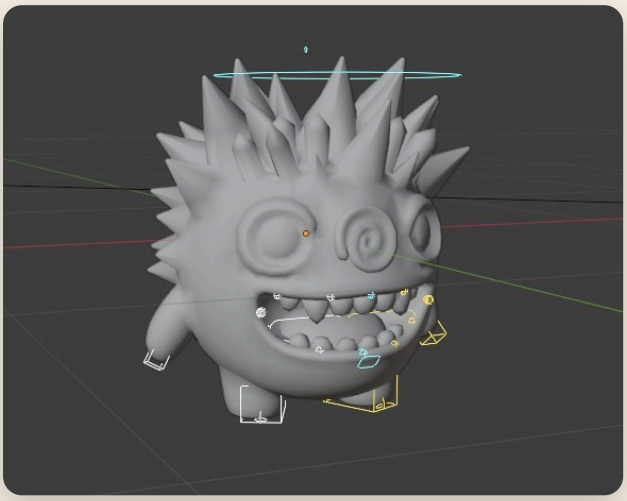
One Animation Blueprint drives walking, jumps, double jump, armed and unarmed stances and the guard, with smooth switches between them. Attacks are montages split into sections, so combos can be chained.



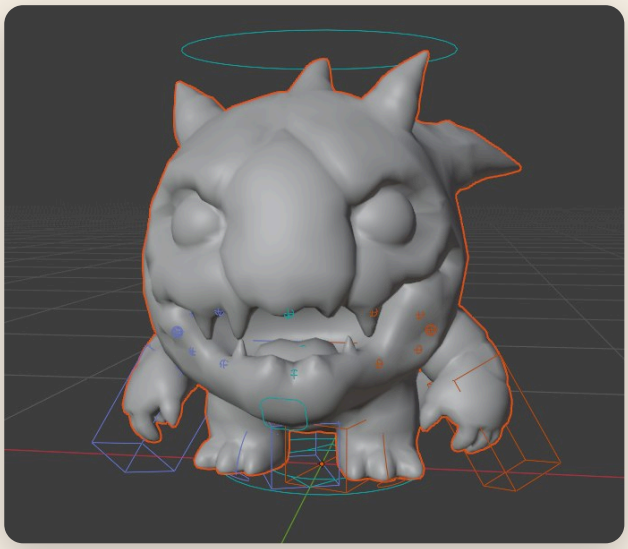
The angel's rig in Blender.

The enemies

Rigged with **Auto-Rig Pro**; I tuned the weights by hand, fixing the traps of AI meshes such as duplicated vertices that tear when the body moves. The animations (5 for the Caster, 9 for the Colossus) were written by **Claude** through Python scripts in Blender, under my direction: I described each move, reviewed the renders and asked for changes.



The Caster, rigged in Blender.



The Colossus, rigged in Blender.

Caster 5 clips

Idle	slow breathing, restless mouth
Walk	loop, with jaw movement
Cast	wind-up and overhead throw
Hit	recoil, one foot stepping back
Death	falls backwards, legs kicking

Colossus 9 clips

Idle	heavy breathing
Walk	loop
Hit	slow, laboured stagger
Slap	torso twist and swing
Charge start	crouch and burst forward
Charge run	loop, arms pumping
Stun	stumble and sway after the charge
Shield roar	three bounces, then a roar
Death	topples over

> **!**
>
> **The most fragile part**
>
> Animation is probably the weakest area of the project. A character this far from the standard humanoid, with a huge head, short limbs and a rigid dress, would have deserved the work of a professional animator.

Two systems, *up close*

Everything in the game is built in Blueprints, and there is far more than fits here. I picked two systems to show in detail; for anything else, feel free to write to me.

1 · Interacting with objects

Every interactive object implements one interface, **BPI_Interagibile**, with three functions:

- `GetAzioni` returns the list of actions: key, label, and the name of the action.
- `EseguiAzione` starts one action by name.
- `GetInteractionLocation` returns the point the character should walk to.

The object decides *what* can be done: the crystal offers *Rotate* and *Listen*, and the widget, the keys and the hold ring are filled from that list. The character owns *how* it looks: it walks to the object, and on arrival a switch on the action name plays the matching animation and effects. A new object reuses all of this; a new kind of action adds one case to that switch.



The list of actions comes from the crystal.



One widget, not one per object

A widget on each crystal hid behind the mesh and floated with it. One HUD widget, projected on the object's point, solved both.

Turn, then walk

Turning and walking at once made the character circle the crystal forever when it started from behind. Two phases fixed it.

2 · Targeting and the lunge

In the first playtests the Casters were almost impossible to hit: they flee by design, every hit pushed them back, and the brush swung at empty air. Two pieces changed the fight completely: the character picks a target at every hit of the combo, and the animation stretches to reach it.

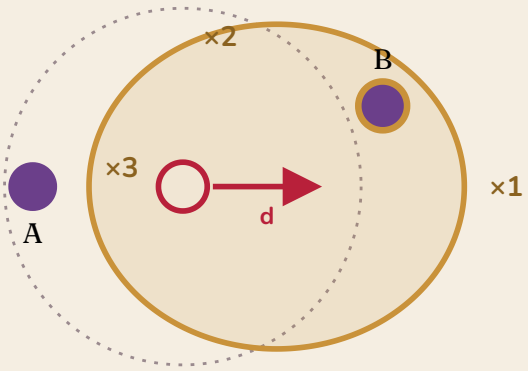
Choosing the target

At every hit, `TrovaBersaglio` gives each living enemy nearby a score and picks the lowest. If the player has locked a target with Tab, that one wins without any search.

$$\mathbf{v}_e = \left(p_e - p_{\text{self}}\right)_{xy}, \qquad \hat{\mathbf{d}} = \begin{cases} \widehat{\text{input}} & \|\text{input}\| > 0.1 \\ \widehat{\text{forward}} & \text{otherwise} \end{cases}$$

$$s(e) = \|\mathbf{v}_e\| \left(2 - \hat{\mathbf{v}}_e \cdot \hat{\mathbf{d}}\right) \qquad e^* = \underset{e \in E}{\arg\min}\ s(e)$$

- **p** are positions: of the player (*self*) and of enemy *e*. $\mathbf{v_e}$ is the vector from the player to the enemy on the ground plane (height ignored); its length is the distance.
- **d** is where the player wants to go: the movement input if the stick is pushed, otherwise the direction the character is facing.
- The **dot product** of the two unit vectors is the cosine of the angle between them: 1 straight ahead, 0 to the side, −1 behind.
- **2 − cos** turns it into a multiplier from 1 (ahead) through 2 (side) to 3 (behind): an enemy behind counts as three times farther away.



Same score everywhere on the gold curve

An ellipse with the player at one focus, stretched along d: it reaches three times farther ahead than behind.

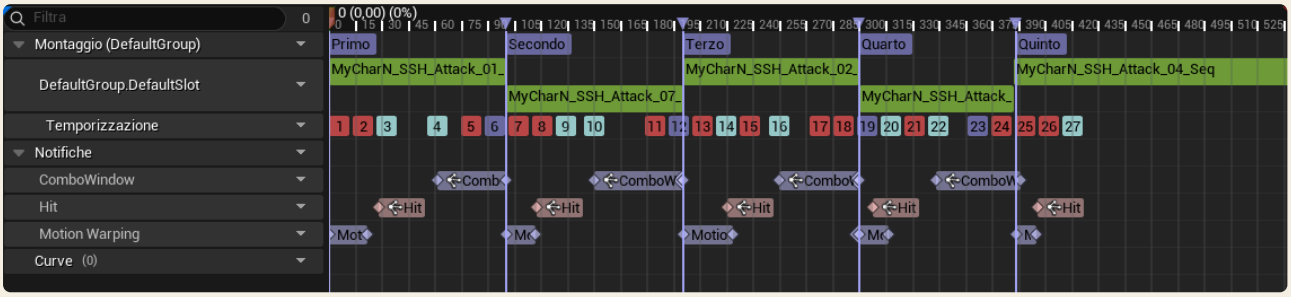
A is closer, but behind: outside.
B is farther, but ahead: inside.

B wins.

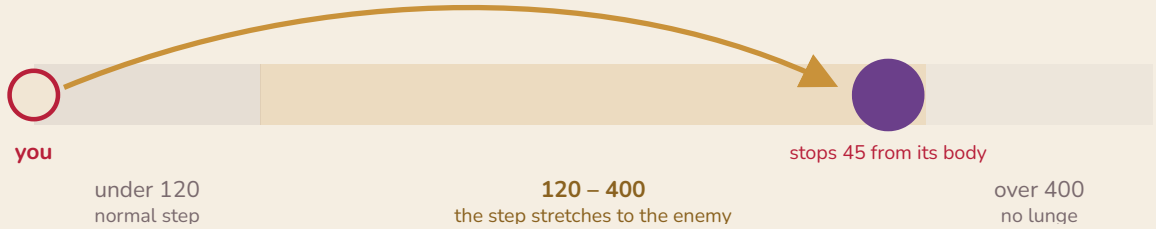
Dashed grey: a plain distance circle, for comparison.

Reaching it: Motion Warping

My first attempt moved the character with a timeline: it looked like a teleport and fought against the animation's own root motion. The solution was to let the animation do the work. Each attack already takes a step forward; with **Motion Warping** that step is stretched until it reaches the enemy.



The five-hit combo montage. On the Motion Warping row, each section has a warping window that ends just before its Hit notify.



Distances are measured to the edge of the enemy's capsule, so the same rule works for the small Caster and the huge Colossus. Only on the ground.



The combo has picked the Caster: the step stretches and the brush lands on target.

From this point on, the fight changed: the combo flows from one enemy to the next instead of chasing them.

Painting with *light*

The effect I am proudest of is the glowing ink that the brush leaves in the air during the special attack: a Niagara ribbon and one custom material.



The special attack: the stroke stays painted in the air, then breaks away.

The ribbon

- **World space:** the trail stays where the brush passed, so it looks painted in the air rather than dragged behind.
- **Constant texture scale:** the UVs tile by length, every 100 units, so a fast swing never stretches the stroke.
- **Life of the ink:** each point lives 2 seconds. It starts dark and see-through, grows brighter and fuller towards the tail (up to four times, in HDR, which triggers the bloom), then detaches all at once instead of fading.



The Niagara emitter.

The material

The painted look comes from three greyscale textures: a brush **stroke**, the **bristles**, and a **noise**. The noise bends the edges so they tear like real ink and digs irregular holes in it; a threshold then turns the soft result into hard edges, like a dry brush.

$$\mathrm{ink}(u) = \mathrm{clamp}\Big(h \cdot \big(S(u)\, B(u) \;-\; D\, N(u) \;-\; \tau \big),\; 0,\; 1 \Big)$$

$$h = 8, \qquad \tau = 0.35, \qquad D \approx 0.82$$

For every point *u* of the trail: S stroke, B bristles, N noise (textures, 0 to 1), D erosion strength, τ threshold, h edge hardness. A simplified form of the material's opacity mask, based on a standard technique, **alpha erosion**. The colour is a dark crimson pushed far above 1, so the bloom turns it into a glow.

Light that arrived late

When the arena lights up, the crystals glow purple and should tint the rocks around them. With Lumen, that bounced light arrived seconds late: emissive surfaces only light the scene through a cache that refreshes a little at a time. I raise the cache refresh rate only for the two seconds the arena is lighting up, then set it back. The glow now reaches the rocks at once, without paying that cost for the rest of the game.

Interface

I set the art direction: an ink veil with a painted porcelain border, the palette and the two typefaces used in this journal. The widgets and textures were built with Claude following those rules. The ink seal fills one vertex for each drop collected: a triangle of circles that fits the mystical, painted look of the game.



A tool, not a *shortcut*

My background is in artificial intelligence, so I used it the way I would use any tool: where it saves time, checked at every step, and never as the author of the decisions.

Area	Tools	What the AI did	What I did
Art	**FLUX.2 [klein], Tripo AI, Hunyuan3D, PixelPasto 2.0**	Clean images, 3D bases, base textures	Drawings, choice and comparison of tools, splitting into parts, assembly in Blender, colour correction
Gameplay	**Claude**	Technical guide: designing systems together, debugging, documentation	Every game Blueprint, built and tuned by hand in the editor
Interface	**Claude Code inside Unreal**	Widgets, UI materials and textures, asset analysis	Art direction, palette and typefaces, review of every asset
Animation	**Claude in Blender**	The enemies' animations via Python scripts	Describing each move, reviewing renders, weight painting, integration in Unreal

How I worked with it

AI tools work best as tools you stay in control of, not as something you ask to build a whole system from scratch. In this project the artistic direction, the technical decisions and the programming always stayed with me.

That let me move fast, alone, without other specialists. It does not replace them: animation, and areas that need deep expertise such as lighting or enemy AI, still call for experienced people.

CHAPTER 7 · LOOKING BACK

What I would *keep*, and change

What worked

- **The art pipeline.** Three months, one person, and a coherent painted world.
- **Targeting and the lunge.** The moment the fight started to feel right.
- **A complete prologue.** Start to finish, with checkpoints, hints, a cinematic and an ending.

What I would change

- **Combat readability.** Enemy intentions are not always clear, and the rhythm drops at times.
- **Animation.** The most fragile part; it needs a professional eye.
- **Sound.** There is none yet, and the fight misses it most.

Next steps

- **C++.** Porting one system, starting with enemy AI, from Blueprints to C++.
- **Sound and feedback.** Audio, and stronger hit feedback in combat.
- **Polishing the arena.** Readability first, then rhythm.

Thank you for reading.

Giovanni Mantegna · gmantegna96@gmail.com

Watch the gameplay